

14. Strong Consistency & Scaling Out: Transaktionen

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

14.1 Vorschau

- Motivation
- Grundlagen
- Pessimistische Sperr-Verfahren
- Optimistische Synchronisierung
- Verteilte Transaktionen
- Zusammenfassung



Motivation: Transaktionen

- Wichtiges Konzept für
 - (Verteilte) Datenbanken
 - Verteilte Caches
 - Verteilte Systeme allgemein
- Bieten Koordinierung
 - Ziel: Effekt des gegenseitigen Ausschlusses, aber mit Nebenläufigkeit
 - Gegenseitiger Ausschluss zu restriktiv → keine Nebenläufigkeit
- Bieten Fehlertoleranz
 - Konsistente Operationsausführung auch bei Ausfällen
 - (Dauerhafte) Speicherung konsistenter Ergebnisse bei Ausfällen



14.2 Grundlagen



- Transaktion = Menge von Operationen
 - Gekennzeichnet mit folgenden Anweisungen
 - **Begin Of Transaction (BOT)**
 - **End Of Transaction (EOT)**
- Abschluss einer Transaktion (TA)
 - **Commit**: erfolgreicher Abschluss → Änderungen werden sichtbar
 - **Abort**: konfliktbedingter Abbruch → alle Änderungen zurücknehmen/verwerfen

ACID Eigenschaften (Härder & Reuter, 1983)

- **Atomicity (atomare Durchführung)**
 - Transaktion wird vollständig oder gar nicht ausgeführt
- **Consistency (Konsistenz der Daten)**
 - Vor und nach der Transaktion ist ein konsistenter Zustand
 - Beispiel: keine ungültigen Referenzen ...
- **Isolation (Isolierung)**
 - Überlappende TAs beeinflussen sich nicht gegenseitig
 - Zwischenzustände einer TA sind nach außen nicht sichtbar
- **Durability (Dauerhaftigkeit)**
 - Resultate einer erfolgreichen TA werden auf Disk gespeichert
 - (überleben einen Systemabsturz)



Nebenläufigkeitskontrolle (eng. concurrency control)

- Ist die Aufgabe des **Transaktionsmanagers**
 - verwaltet Transaktionen
 - kontrolliert Datenzugriffe der Operationen aller Transaktionen
 - **entscheidet über die Ausführungsreihenfolge der Operationen**
- Zwei Kategorien von Nebenläufigkeitskontrolle:
 - **Pessimistische Verfahren mit Sperren**
 - **Optimistische Synchronisierung ohne Sperren**



- Ablaufplan (engl. schedule): Abfolge der Operationen mehrerer überlappender Transaktionen
- Ziel: Konfliktfreien Schedule finden, der äquivalent zu einer seriellen Ausführung dieser Transaktionen ist.
 - gesucht konfliktfreier Schedule, mit maximaler Nebenläufigkeit
- Mögliche Konflikte bei überlappenden Transaktionen:
 - $\text{read}_{T_1} \text{ — } \text{read}_{T_2}$: kein Konflikt (unabhängig von der Ausführungsreihenfolge)
 - $\text{read}_{T_1} \text{ — } \text{write}_{T_2}$: Konflikt: entweder alter oder neuer Wert gelesen
 - $\text{write}_{T_1} \text{ — } \text{write}_{T_2}$: Konflikt: welcher Wert ist im Endergebnis?

Beispiel (1): Transaktionen

Überweisungstransaktion T_1

BOT_{T_1}

```
k1 = read(Konto1);
```

```
write(Konto1, k1 - Überweisung);
```

```
k2 = read(Konto2);
```

```
write(Konto2, k2 + Überweisung);
```

$COMMIT_{T_1}$

Summentransaktion T_2

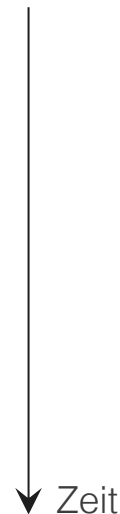
BOT_{T_2}

```
a = read(Konto1);
```

```
b = read(Konto2);
```

```
sum = a + b;
```

$COMMIT_{T_2}$



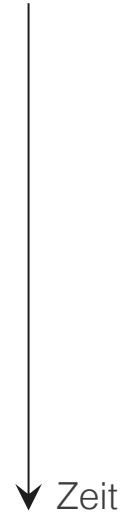
Beispiel (2): Transaktionen vereinfachte Darstellung

Überweisungstransaktion T_1

```
BOTT1  
read(K1) ;  
write(K1) ;  
read(K2) ;  
write(K2) ;  
COMMITT1
```

Summentransaktion T_2

```
BOTT2  
read(K1) ;  
read(K2) ;  
COMMITT2
```



Beispiel (3): Serielles Schedule

Anfangszustand: $K_1 = \$500$, $K_2 = \$700$; Dann überweist T_1 \$200 von K_1 nach K_2

Schedule 1: $T_1 \rightarrow T_2$

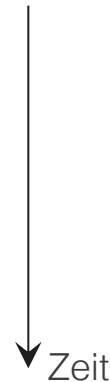
```
BOTT1  
read( $K_1$ ) ;  
write( $K_1$ ) ;  
read( $K_2$ ) ;  
write( $K_2$ ) ;  
COMMITT1
```

```
BOTT2  
read( $K_1$ ) ;  
read( $K_2$ ) ;  
COMMITT2
```

Schedule 2: $T_2 \rightarrow T_1$

```
BOTT2  
read( $K_1$ ) ;  
read( $K_2$ ) ;  
COMMITT2
```

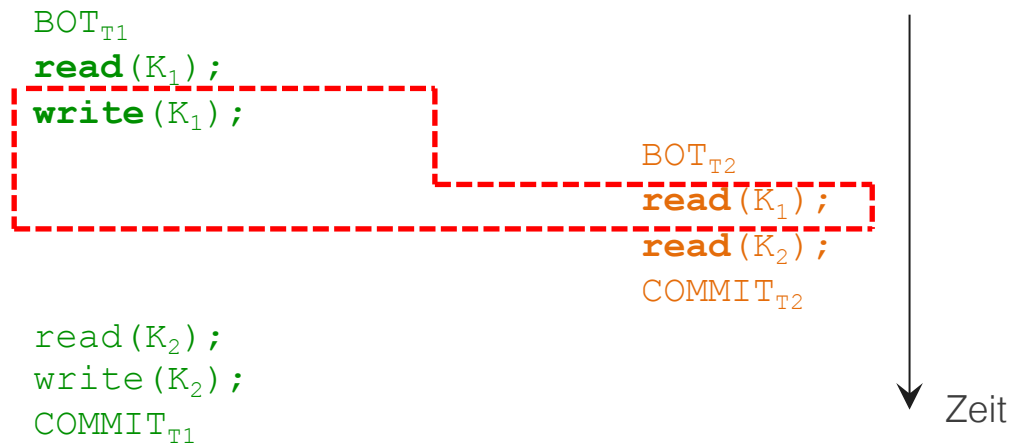
```
BOTT1  
read( $K_1$ ) ;  
write( $K_1$ ) ;  
read( $K_2$ ) ;  
write( $K_2$ ) ;  
COMMITT1
```



- Beide Schedules sind konfliktfrei und liefern das gleiche Ergebnis:
 - T_1 : Konto1 $\rightarrow$ \$300, Konto2 $\rightarrow$ \$900; T_2 : Summe $\rightarrow$ \$1200

Beispiel (4): Ungültiges nebenläufiges Schedule

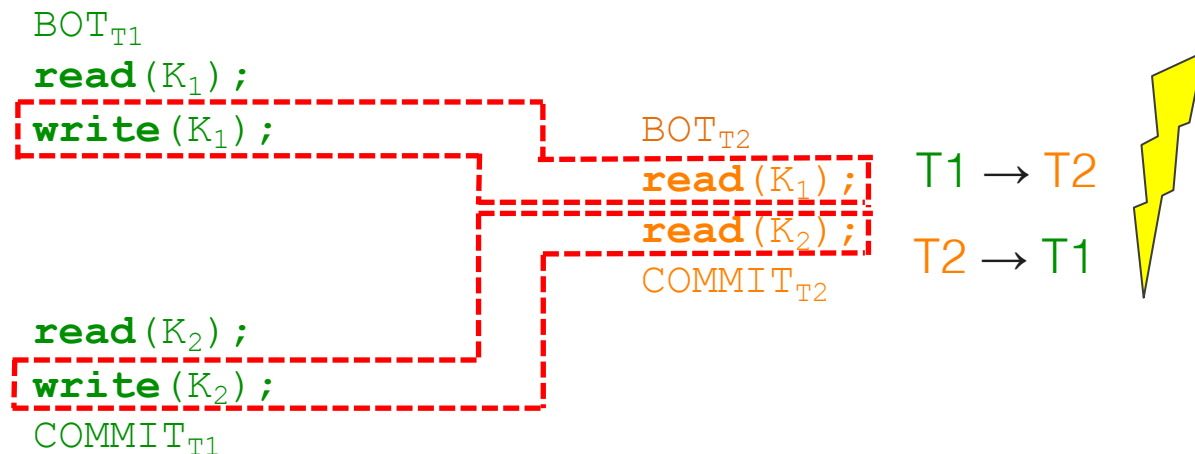
Anfangszustand: $K_1 = \$500$, $K_2 = \$700$; Dann überweist T_1 \$200 von K_1 nach K_2



- T_2 liest K_1 vor dem Commit von T_1 bestätigt sind $\rightarrow$ Isolation verletzt
 - T_1 : Konto1 $\rightarrow$ \$300, Konto2 $\rightarrow$ \$900 $\rightarrow$ OK
 - T_2 : Summe $\rightarrow$ \$1000 $\rightarrow$ **ERROR**

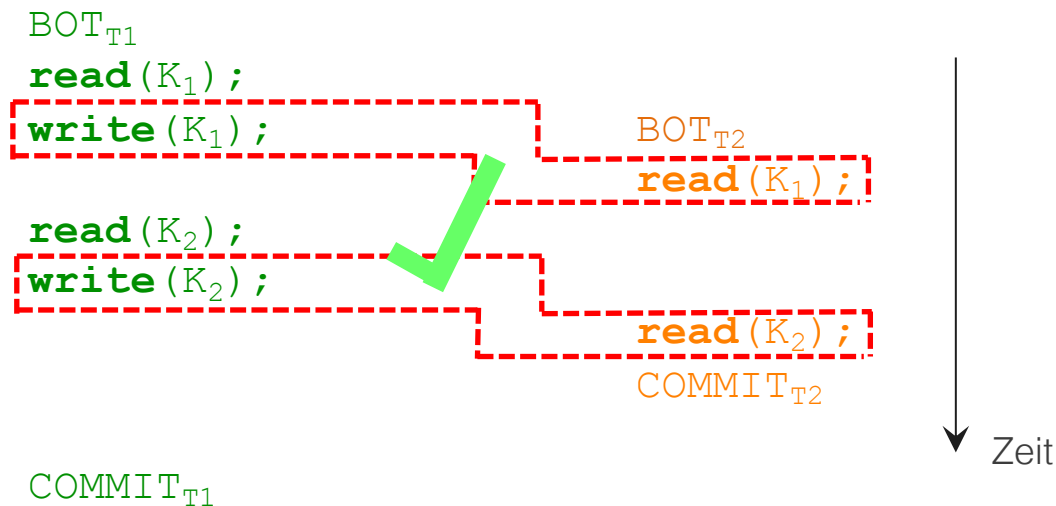
Bedingung für die Serialisierbarkeit

- Zwei Transaktionen sind serialisierbar, wenn alle in Konflikt stehenden Zugriffspaare dieser Transaktionen in der gleichen Reihenfolge ausgeführt werden.
- Im letzten Beispiel galt das nicht:



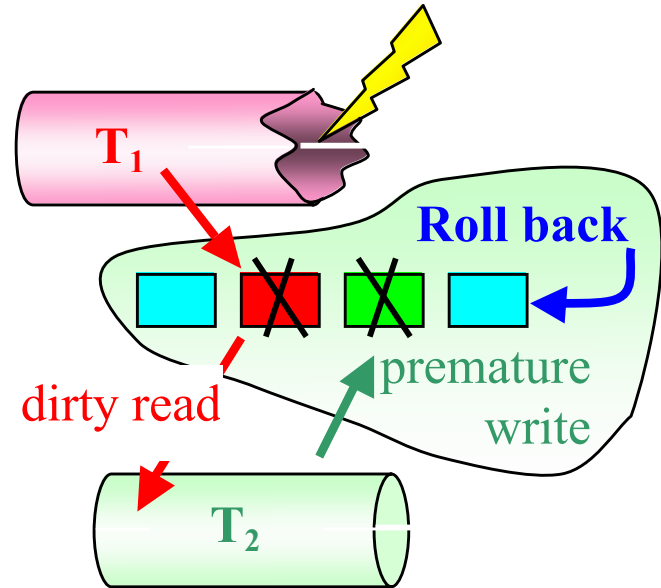
Bedingung für die Serialisierbarkeit

- Zwei Transaktionen sind serialisierbar, wenn alle in Konflikt stehenden Zugriffspaare dieser Transaktionen in der gleichen Reihenfolge ausgeführt werden.
- Diese nebenläufige Ausführung erfüllt die Serialisierbarkeit



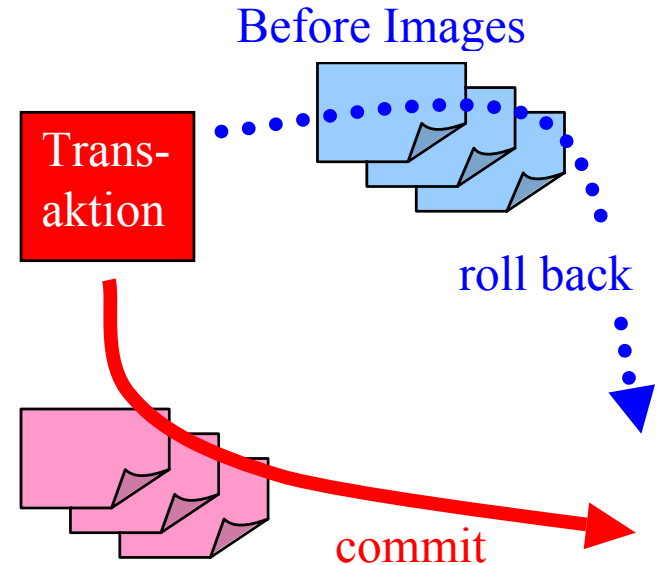
Probleme bei Transaktionsabbrüchen

- Probleme trotz Serialisierbarkeit
- **"Dirty Read":**
 - T_2 liest Daten, die später wieder ungültig werden.
- **"Premature Write":**
 - verfrühtes Schreiben von T_2 wird durch eine fremde Rücksetzung überschrieben (hier T_1).
- Lösung: T_2 wird in die Rücksetzung von T_1 hineingezogen → „cascading abort“
 - Generell müssen Transaktionen rücksetzbar sein



Rücksetzbarkeit von Transaktionen

- Atomarität einer Transaktion:
 - Transaktion komplett oder gar nicht durchführen
 - Änderungen können rückgängig gemacht werden, mit Hilfe von "before images" (Schattenkopien).
- Falls Commit:
 - Erfolgreich: Änderungen permanent machen und „before images“ wegwerfen
 - Nicht erfolgreich: Rollback mit Schattenkopien.



14.3 Pessimistische Sperr-Verfahren



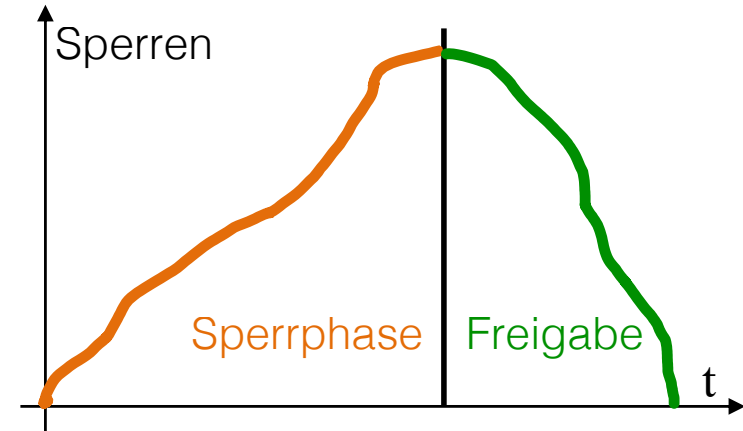
Pessimistische Sperr-Verfahren

- Annahme: Konflikte zw. Transaktionen treten relativ häufig auf.
- Ziel: Verhindern des Konfliktfalls durch den Einsatz von Sperren (engl. locks)
- Beispiel:
 - $T1 = \{ a.lock, a.read, a.unlock, b.lock, b.write, b.unlock \}$
 $T2 = \{ b.lock, b.write, b.unlock, a.lock, a.write, a.unlock \}$
 - möglicher Ablauf:
 $a.lock, a.read, b.lock, a.unlock, b.write, b.unlock, b.lock, b.write, a.lock, a.write, b.unlock, a.unlock$
 - Ergebnis:
 $a.read_{T1}$ vor $a.write_{T2}$ aber $b.write_{T1}$ nach $b.write_{T2}$
→ Konflikte (trotz Sperren): $read_{T1} - write_{T2}$ und $write_{T2} - write_{T1}$
 - Lösung: nach Sperrfreigabe dürfen keine Sperren mehr angefordert werden.



2-Phasen Sperrprotokoll

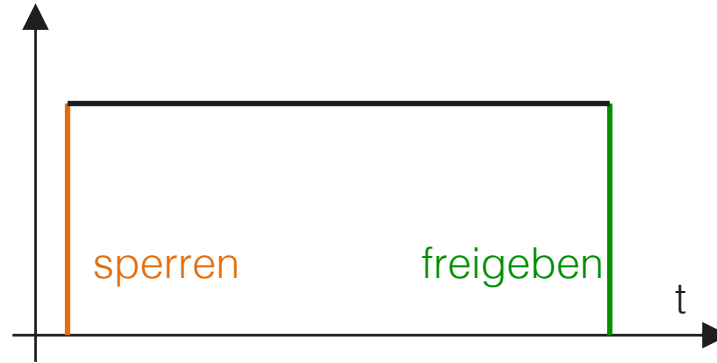
- Garantiert Serialisierbarkeit
 - Erste Phase: Anfordern von Sperren
 - Zweite Phase: Freigeben von Sperren



- Nachteil:
 - Gefahr von Cascading-Aborts bei Abbruch in 2. Phase (wegen Dirty-Reads)
 - Verklemmungen können auftreten
 - mögliche Lösung: Ordnung auf Sperren definieren

Striktes 2-Phasen Sperrprotokoll

- Alle Sperren zu Beginn anfordern und am Ende freigeben



- Bewertung:
 - Keine Verklemmungen möglich
 - Abbruch vor Freigabe führt nicht zu Folgeabbrüchen
 - Aber alle benötigten Sperren müssen vorab bekannt sein

Pessimistische Sperr-Verfahren - Bemerkungen

- Sperren über ein Netzwerk anzufordern, ist teuer
- Und Sperren erst am Ende der Transaktion freizugeben schränkt die Nebenläufigkeit stark ein.
- Aber dennoch sinnvoll bei einer hohen Konfliktwahrscheinlichkeit
- Viele weitere Sperrverfahren in Datenbanken vorhanden



14.4 Optimistische Synchronisierung



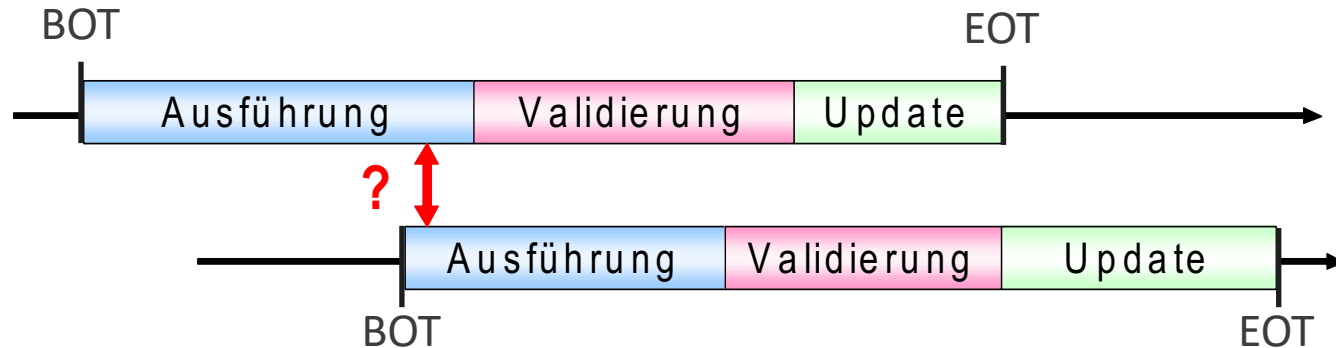
Optimistische Synchronisierung

- Annahme: Konflikte zw. Transaktionen treten selten auf.
 - Ferner müssen alle Transaktionen rücksetzbar sein (Schattenkopien bei Schreibzugriffen)
- Hier arbeiten die Transaktionen ohne Sperren:
 - Damit sind auch keine Verklemmungen möglich
 - Und der Aufwand für das Anfordern und Freigeben von Sperren entfällt



Transaktionen unterteilt in drei Phasen

- **Ausführung:**
 - TA arbeitet nebenläufig auf lokalen Kopien,
 - Aufzeichnung des Read- und Write-Sets (+ Schattenkopien).
- **Validierung:** prüft auf Konflikte zwischen überlappenden TAs
- **Update:** macht Änderungen bekannt (und persistent)



Validierung

- Wichtig: Validierungsphase erfolgt im gegenseitigen Ausschluss
 - Zu einem Zeitpunkt kann immer nur eine TA validiert werden
 - Dafür wird meist ein zentraler Transaktionsmanager verwendet
- Validierung erfolgt mit Hilfe aufgezeichneter Read- und Write-Sets
- Zuteilung von aufsteigenden Transaktionsnummern zu Beginn der Validierung
- Serialisierung anhand der Transaktionsnummern



Rückwärtsvalidierung (1)

- finished_set Menge der abgeschlossenen Transaktionen
- $\text{r_set}(T_i)$ Menge der von T_i gelesenen Datenelemente
- $\text{w_set}(T_i)$ Menge der von T_i geschriebenen Datenelemente

- $\text{start_time}(T_i)$ Startzeitpunkt der Transaktion T_i
- $\text{end_time}(T_i)$ Endzeitpunkt der Transaktion T_i



Rückwärtsvalidierung (2)

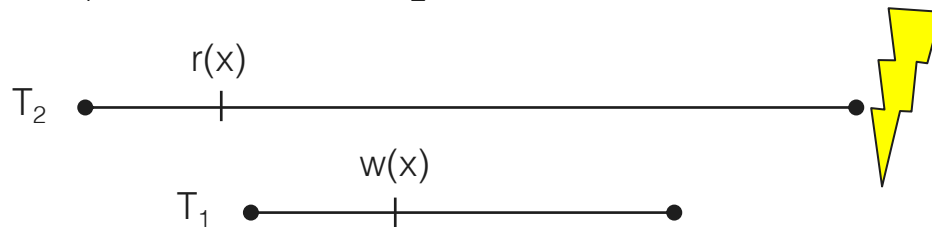
```
validate_backward( $T_i$ )  
begin  
   $conflict = false$ ;  
  for all  $T_j \in finished\_set \wedge j \neq i \wedge end\_time(T_j) > start\_time(T_i)$  do  
    if  $(r\_set(T_i) \cap w\_set(T_j) \neq \emptyset)$  then  
       $conflict = true$ ;  
    end do;  
  if  $conflict$  then abort( $T_i$ ) else commit( $T_i$ )  
end validate_backward;
```



Rückwärtsvalidierung: Beispiele

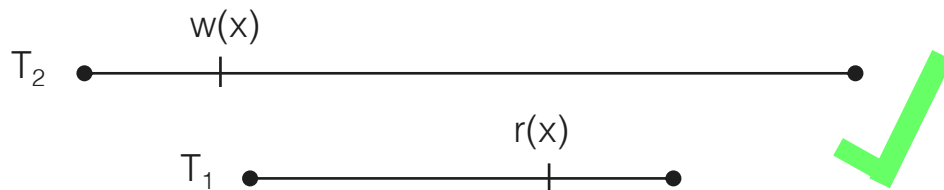
- Beispiel 1: Konflikt

- T_2 validiert sich gegen $T_1 \rightarrow$ Konflikt, da T_2 veralteten Wert liest



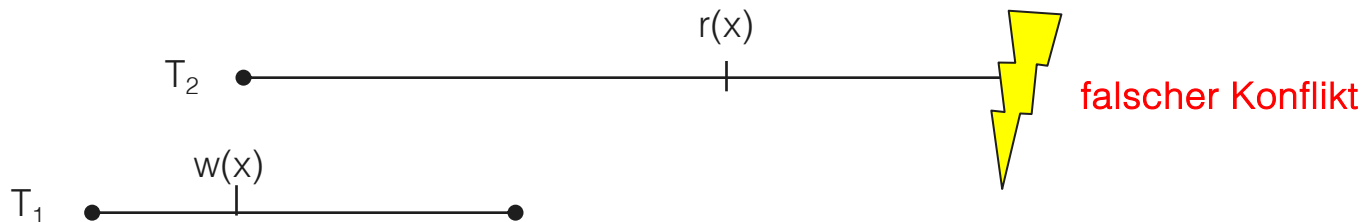
- Beispiel 2: kein Konflikt

- T_2 validiert sich gegen $T_1 \rightarrow$ kein Konflikt
- $w(x)$ von T_2 wird erst beim Commit sichtbar



Rückwärtsvalidierung: Bemerkungen

- Unter Umständen müssen Write-Sets lange aufbewahrt werden
- Es kann immer nur die validierende Transaktion abgebrochen werden. Bereits abgeschlossene TAs nicht.
- Es kommt zu unnötigen Abbrüchen, auch wenn Transaktionen neuere Versionen gelesen haben → Beispiel: $r(x)$ nach Commit von T_1



Rückwärtsvalidierung mit Versionsnummern

- Verbesserung: globale Commit-Nummer
 - Wird vom Transaktionsmanager aufsteigend vergeben, für jede Transaktion, bei deren Commit
 - Wird mit dem Write-Set jeder Transaktion mit abgespeichert und bei der Validierung berücksichtigt
 - Für jeden Lese-Zugriff wird gespeichert, welche Version (Commit-Nummer) gelesen wurde → damit können falsche Konflikte vermieden werden

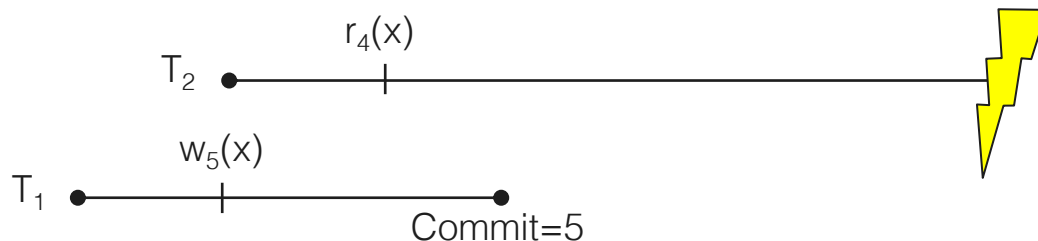
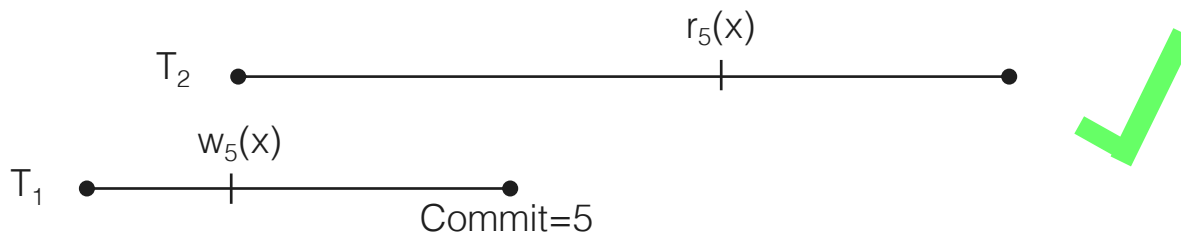


Rückwärtsvalidierung mit Versionsnummern

- Verbesserung: globale Commit-Nummer
- Beispiel:

$r_i(x), w_i(x)$

i = Commit-Nummer



Vorwärtsvalidierung (2)

- `active_set` Menge der aktiven Transaktionen
- `r_set( $T_i$ )` Menge der von T_i gelesenen Datenelemente
- `w_set( $T_i$ )` Menge der von T_i geschriebenen Datenelemente

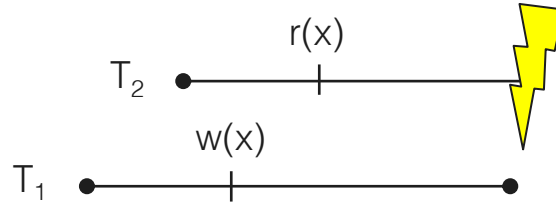
```
validate_forward( $T_i$ )
begin
    conflict = false;
    for all  $T_j \in active\_set \wedge j \neq i$  do
        if  $(r\_set(T_j) \cap w\_set(T_i) \neq \emptyset)$  then
            conflict = true;
        end do;
    if conflict then abort( $T_i$ ) else commit( $T_i$ )
end validate_forward;
```



Vorwärtsvalidierung: Beispiele

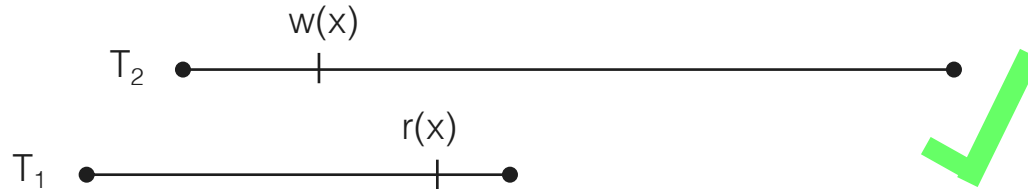
- Beispiel 1: Konflikt

- T_1 validiert sich gegen $T_2 \rightarrow$ Konflikt, da T_2 veralteten Wert liest



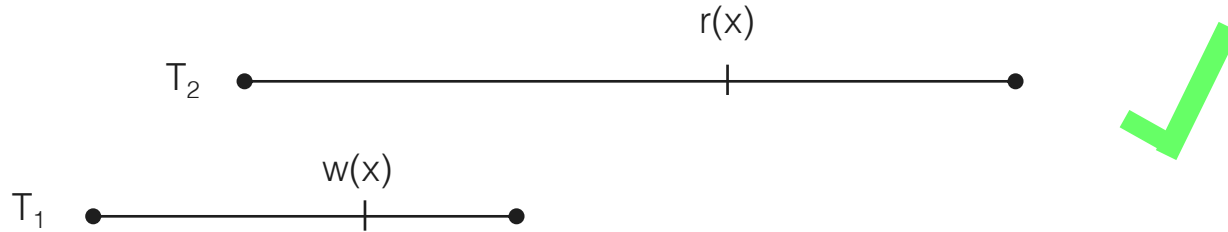
- Beispiel 2: kein Konflikt

- T_1 validiert sich gegen $T_2 \rightarrow$ kein Konflikt
- $w(x)$ von T_2 wird erst beim Commit sichtbar



Vorwärtsvalidierung: Bemerkungen

- Im Gegensatz zur Rückwärtsvalidierung entsteht hier kein falscher Konflikt
 - Wenn sich T_1 validiert, ist der `read_set` leer



$r_i(x)$
 i = Commit-Nummer
→ Wann wurde geschrieben?

Vorwärtsvalidierung: Bemerkungen

- Vorteil: Es kann ausgewählt werden, welche Transaktion abgebrochen werden soll.
- Optimistische Synchronisierung ist besser als sperrbasierte Verfahren, wenn wenig Konflikte eintreten



14.5 Verteilte Transaktionen



Verteilte Transaktionen

- Hier sind die Operationen einer Transaktion verteilt über mehrere Server
 - Notwendig, falls eine Tabelle einer Datenbank aufgeteilt und auf mehreren Servern gespeichert wird
- Unterschiede: verteilte Transaktion vs Transaktion in einem verteilten System
- Problem: verteilte Sicherstellung der Serialisierbarkeit
 - lokale Auswertung auf einem Server kann zum Abbruch der gesamten verteilten Transaktion führen
 - beim Commit muss eine Auswertung auf allen Servern erfolgen, auf denen Teile einer Transaktion ausgeführt wurden.

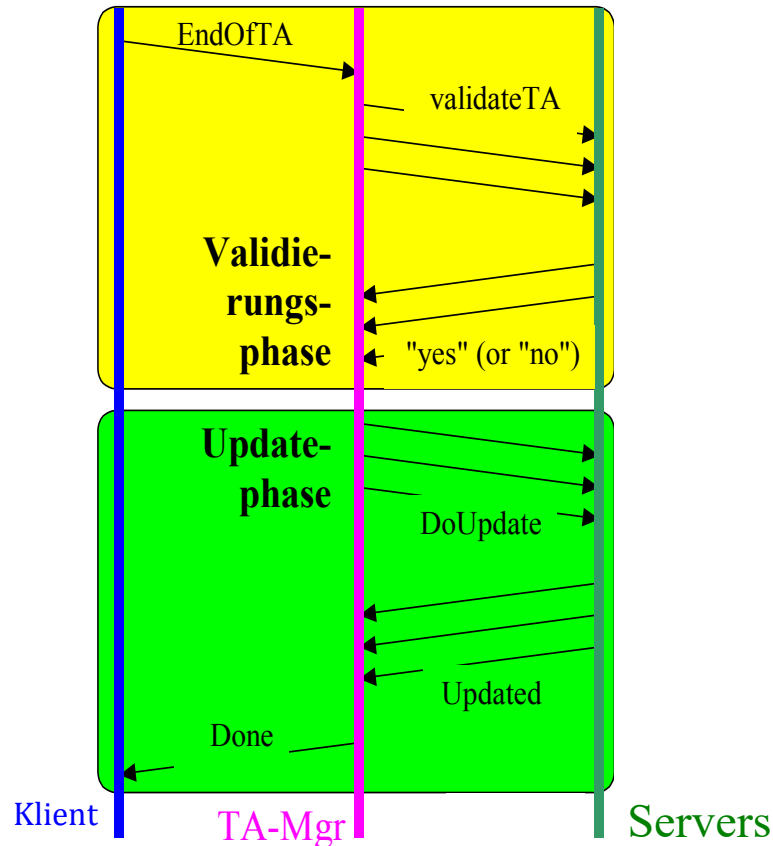


Verteilte Transaktionen - Zwei-Phasen Commit

- Unterteilung in zwei Phasen: Validierungs- und Update-Phase.
- Commit-Request am Ende der Transaktion:
 - Client sendet EndOfTA zum Transaktionsmanager (TA-Manager)
 - 1. Phase: Validierung
 - TA-Manager fordert alle beteiligten Server auf, eine Validierung durchzuführen und das Ergebnis zurückzumelden
 - Abort: falls ein oder mehr Knoten einen Konflikt erkennen oder ein Timeout eintritt
 - Ansonsten 2. Phase
 - 2. Phase: Update
 - TA-Manager fordert alle beteiligten Knoten auf das Update durchzuführen
 - Wenn alle das Update bestätigt hat, meldet der TA-Manager den Erfolg an den Client



Verteilte Transaktionen - Zwei-Phasen Commit



Fehlerfälle: Server stürzt ab

- 1. Fall: Ein Server stürzt in der Validierungsphase ab, bevor er seine Antwort zum TA-Manager schickt
 - TA-Manager bricht diese Transaktion ab
 - (unproblematischer Fall)
- 2. Fall: Server hat mit „Yes“ geantwortet, stürzt dann aber ab
 - Wenn der Server wiedergestartet ist, kontaktiert er den TA-Manager, um den Ausgang der TA zu erfahren
 - TA-Manager muss dafür das Ergebnis der Validierungsphase in einem Log speichern, bevor er Nachrichten in der Updatephase verschickt
 - TA-Manager blockiert und wartet auf das Recovery des Servers



Fehlerfälle: Server stürzt ab

- 3. Fall: Server hat „DoUpdate“ empfangen und stürzt dann ab
 - Der Server muss dafür lokal auch ein Log haben
 - Damit kann er nach dem Neustart erkennen, dass er noch antworten muss und ggf. die Transaktion lokal durchführen muss
 - TA-Manager blockiert und wartet auf das Recovery des Servers



Fehlerfälle: TA-Manager stürzt ab

- TA-Manager verwendet sein Log, um seinen Zustand nach dem Neustart wiederherzustellen und eine Phase ggf. zu beenden
- 1. Fall: TA-Manager stürzt in der Validierungsphase ab
 - Sobald ein Server in der Validierungsphase mit „Yes“ geantwortet hat, muss er warten bis das Ergebnis vom TA-Manager vorliegt, solange ist er blockiert
 - Der TA-Manager muss bei Bedarf die Phase nach einem Neustart beenden
- 2. Fall: TA-Manager stürzt in der Updatephase ab
 - Sobald ein Server „DoUpdate“ erhalten hat, kann er commiten und weiterarbeiten, andernfalls ist er blockiert
 - Der TA-Manager muss bei Bedarf die Phase nach einem Neustart beenden



Fehlerfälle: TA-Manager stürzt ab

- Wenn der TA-Manager nicht neugestartet werden kann oder das Log nicht verfügbar ist, sind alle Server blockiert
 - Die Server könnten versuchen sich untereinander auszutauschen, um den Ausgang der letzten TA zu erfahren.
 - Aber was ist, wenn alle bis auf einen antworten? Auch dann geht es nicht weiter
- Eine Verbesserung ist das 3 Phasen Commit Protokoll (3 PC), welches die Blockierungen vermeidet. Dieses Protokoll besprechen wir in dieser Veranstaltung nicht.



14.6 Zusammenfassung



Zusammenfassung

- Transaktionen bieten eine strenge Konsistenz, skalieren aber nur bedingt
 - Werden daher nur dort verwendet wo unbedingt eine strenge Konsistenz erforderlich ist
- Sperrverfahren sind sinnvoll, falls viel geschrieben wird, andernfalls ist eine optimistische Synchronisierung vorteilhaft
- Ein 2-Phase-Commit ist für verteilte Transaktionen notwendig, aber langsam (durch die Multicasts)

